

Nama : Gathan Abramovic
NPM : 251063250063
Kelas : 2C
Mata Kuliah : Algoritma Struktur Data

1. Buble Sort

```
1 public class BubbleSortNilai {  
2  
3     public static void main(String[] args) {  
4  
5         int[] nilai = {85, 72, 93, 60, 78, 95, 88, 65, 74, 81};  
6  
7         int n = nilai.length;  
8         int perbandingan = 0;  
9         int pertukaran = 0;  
10  
11         System.out.println("Nilai awal:");  
12         for (int x : nilai) {  
13             System.out.print(x + " ");  
14         }  
15         System.out.println("\n");  
16  
17         long start = System.nanoTime();  
18  
19         // Bubble Sort descending  
20         for (int i = 0; i < n - 1; i++) {  
21  
22             System.out.println("Iterasi ke-" + (i + 1));  
23  
24             for (int j = 0; j < n - i - 1; j++) {  
25                 perbandingan++;  
26  
27                 if (nilai[j] < nilai[j + 1]) {  
28                     int temp = nilai[j];  
29                     nilai[j] = nilai[j + 1];  
30                     nilai[j + 1] = temp;  
31                     pertukaran++;  
32                 }  
33             }  
34  
35             // tampilkan tiap iterasi  
36             for (int x : nilai) {  
37                 System.out.print(x + " ");  
38             }  
39             System.out.println("\n");  
40         }  
41  
42         long end = System.nanoTime();  
43     }  
}
```

```

1  System.out.println("Nilai terurut:");
2      for (int x : nilai) {
3          System.out.print(x + " ");
4      }
5
6      System.out.println("\nJumlah perbandingan: " + perbandingan);
7      System.out.println("Jumlah pertukaran: " + pertukaran);
8      System.out.println("Waktu eksekusi: " + (end - start) + " ns");
9  }
10 }

```

```

Nilai awal:
85 72 93 60 78 95 88 65 74 81

Iterasi ke-1
85 93 72 78 95 88 65 74 81 60

Iterasi ke-2
93 85 78 95 88 72 74 81 65 60

Iterasi ke-3
93 85 95 88 78 74 81 72 65 60

Iterasi ke-4
93 95 88 85 78 81 74 72 65 60

Iterasi ke-5
95 93 88 85 81 78 74 72 65 60

Iterasi ke-6
95 93 88 85 81 78 74 72 65 60

Iterasi ke-7
95 93 88 85 81 78 74 72 65 60

Iterasi ke-8
95 93 88 85 81 78 74 72 65 60

Iterasi ke-9
95 93 88 85 81 78 74 72 65 60

Nilai terurut:
95 93 88 85 81 78 74 72 65 60
Jumlah perbandingan: 45
Jumlah pertukaran: 22
Waktu eksekusi: 6904100 ns

```

Output Program

- a. Pengurutan nilai dari yang tertinggi ke terendah dilakukan dengan membandingkan setiap pasangan nilai yang bersebelahan dalam array. Jika nilai di sebelah kiri lebih kecil dibandingkan dengan nilai di sebelah kanan, maka kedua nilai tersebut akan ditukar posisinya. Proses ini bertujuan agar nilai yang lebih besar bergerak ke bagian depan array. Langkah tersebut dilakukan secara berulang dalam beberapa iterasi hingga seluruh nilai tersusun secara urut dari yang terbesar hingga yang terkecil. Dengan demikian, nilai tertinggi akan berada di awal dan nilai terendah berada di akhir.

- b. Program menampilkan proses pengurutan pada setiap iterasi untuk memperlihatkan perubahan posisi nilai dalam array. Pada setiap iterasi luar (loop pertama), program akan mencetak isi array setelah proses perbandingan dan pertukaran selesai dilakukan. Hal ini bertujuan agar pengguna dapat melihat bagaimana nilai-nilai berpindah secara bertahap hingga akhirnya tersusun dari yang tertinggi ke terendah.

```
Iterasi ke-1
85 93 72 78 95 88 65 74 81 60

Iterasi ke-2
93 85 78 95 88 72 74 81 65 60
```

- c. Program ini menghitung jumlah perbandingan serta pertukaran yang berlangsung selama pengurutan menggunakan metode Bubble Sort. Setiap kali dua elemen dalam array diperbandingkan, jumlah perbandingan akan meningkat satu. Apabila terjadi pertukaran antara dua nilai, maka jumlah pertukaran juga akan bertambah. Perhitungan ini bertujuan untuk mengukur seberapa banyak operasi yang dilakukan oleh algoritma selama proses pengurutan.

```
Nilai terurut:
95 93 88 85 81 78 74 72 65 60
Jumlah perbandingan: 45
Jumlah pertukaran: 22
```

- d. Program menampilkan waktu eksekusi untuk mengetahui seberapa lama proses pengurutan dilakukan oleh algoritma Bubble Sort. Waktu eksekusi dihitung dengan mencatat waktu sebelum proses sorting dimulai dan setelah proses selesai menggunakan `System.nanoTime()`. Selisih antara kedua waktu tersebut menunjukkan durasi yang dibutuhkan program dalam melakukan pengurutan.

2. Selection sort

Program menggunakan algoritma Selection Sort untuk mengurutkan data secara ascending. Pada setiap iterasi, program mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan posisi saat ini. Proses ini diulang hingga seluruh data terurut, dan setiap perubahan ditampilkan pada setiap iterasi.



```
1 public class SelectionSortDemo {
2
3     public static void main(String[] args) {
4
5         int[] data = {99, 98, 3, 2, 5, 4, 70, 50};
6
7         int n = data.length;
8
9         System.out.println("Array sebelum diurutkan:");
10        for (int x : data) {
11            System.out.print(x + " ");
12        }
13        System.out.println("\n");
14
15        // Selection Sort (ascending)
16        for (int i = 0; i < n - 1; i++) {
17
18            int minIndex = i;
19
20            // cari nilai terkecil
21            for (int j = i + 1; j < n; j++) {
22                if (data[j] < data[minIndex]) {
23                    minIndex = j;
24                }
25            }
26
27            // tukar
28            int temp = data[i];
29            data[i] = data[minIndex];
30            data[minIndex] = temp;
31
32            // tampilkan setiap iterasi
33            System.out.print("Iterasi ke-" + (i + 1) + ": ");
34            for (int x : data) {
35                System.out.print(x + " ");
36            }
37            System.out.println();
38        }
39
40        System.out.println("\nArray setelah diurutkan:");
41        for (int x : data) {
42            System.out.print(x + " ");
43        }
44    }
45 }
```

Array sebelum diurutkan:

99 98 3 2 5 4 70 50

Iterasi ke-1: 2 98 3 99 5 4 70 50

Iterasi ke-2: 2 3 98 99 5 4 70 50

Iterasi ke-3: 2 3 4 99 5 98 70 50

Iterasi ke-4: 2 3 4 5 99 98 70 50

Iterasi ke-5: 2 3 4 5 50 98 70 99

Iterasi ke-6: 2 3 4 5 50 70 98 99

Iterasi ke-7: 2 3 4 5 50 70 98 99

Array setelah diurutkan:

2 3 4 5 50 70 98 99

Pada iterasi ke-3, Selection Sort sudah menempatkan tiga nilai terkecil di depan, yaitu 2, 3, dan 4. Proses ini dilakukan dengan mencari nilai minimum dari sisa array dan menukarnya ke posisi yang sesuai.

3. Insertion Sort

Array sebelum diurutkan:

99 98 3 2 5 4 70 50

Iterasi ke-1: 2 98 3 99 5 4 70 50

Iterasi ke-2: 2 3 98 99 5 4 70 50

Iterasi ke-3: 2 3 4 99 5 98 70 50

Iterasi ke-4: 2 3 4 5 99 98 70 50

Iterasi ke-5: 2 3 4 5 50 98 70 99

Iterasi ke-6: 2 3 4 5 50 70 98 99

Iterasi ke-7: 2 3 4 5 50 70 98 99

Array setelah diurutkan:

2 3 4 5 50 70 98 99

2 3 98 99 5 4 70 50

4. Searching Sort

Kode Program Searching Sort, program memasukan nilai yang ingin di cari, mencari nilai menggunakan Binary, program juga menampilkan indeks dari nilai yang ditemukan serta waktu eksekusi masing-masing algoritma menggunakan *System.nanoTime()*.

```

1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class CariNilaiMahasiswa {
5
6      public static void main(String[] args) {
7
8          int[] nilai = {85, 72, 90, 65, 78, 95, 88, 60, 82, 75};
9
10         Scanner input = new Scanner(System.in);
11         System.out.print("Masukkan nilai yang ingin dicari: ");
12         int cari = input.nextInt();
13
14         // =====
15         // LINEAR SEARCH
16         // =====
17         long startLinear = System.nanoTime();
18
19         int indexLinear = -1;
20         for (int i = 0; i < nilai.length; i++) {
21             if (nilai[i] == cari) {
22                 indexLinear = i;
23                 break;
24             }
25         }
26
27         long endLinear = System.nanoTime();
28
29         System.out.println("\n=== Linear Search ===");
30         if (indexLinear != -1) {
31             System.out.println("Nilai ditemukan di index: " +
indexLinear);
32         } else {
33             System.out.println("Nilai tidak ditemukan");
34         }
35         System.out.println("Waktu eksekusi: " + (endLinear -
startLinear) + " ns");
36
37         // =====
38         // BINARY SEARCH
39         // =====
40         Arrays.sort(nilai); // wajib urutkan dulu
41
42         long startBinary = System.nanoTime();
43
44         int kiri = 0;
45         int kanan = nilai.length - 1;
46         int indexBinary = -1;
47
48         while (kiri <= kanan) {
49             int mid = (kiri + kanan) / 2;
50
51             if (nilai[mid] == cari) {
52                 indexBinary = mid;
53                 break;
54             } else if (nilai[mid] < cari) {
55                 kiri = mid + 1;
56             } else {
57                 kanan = mid - 1;
58             }
59         }
60
61         long endBinary = System.nanoTime();
62
63         System.out.println("\n=== Binary Search ===");
64         if (indexBinary != -1) {
65             System.out.println("Nilai ditemukan di index: " +
indexBinary);
66         } else {
67             System.out.println("Nilai tidak ditemukan");
68         }
69         System.out.println("Waktu eksekusi: " + (endBinary -
startBinary) + " ns");
70
71         input.close();
72     }
73 }

```

```

PS C:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12> cd "c:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12" & java CariNilaiMahasiswa } ; if ($?) { java CariNilaiMahasiswa }
Masukkan nilai yang ingin dicari: 77

=== Linear Search ===
Nilai tidak ditemukan
Waktu eksekusi: 700 ns

=== Binary Search ===
Nilai tidak ditemukan
Waktu eksekusi: 1400 ns
PS C:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12>

```

Disini saya mencoba memasukkan atau mencari nilai mahasiswa 77, lalu program melakukan pencarian menggunakan dua metode, yaitu Linear Search dan Binary Search.

Metode pertama menggunakan Linear Search, program langsung mengecek setiap elemen dalam array satu per satu. Dari proses tersebut, nilai 77 tidak ditemukan di dalam data sehingga program menampilkan “Nilai tidak di temukan” dengan waktu eksekusi 700 ns.

Untuk metode ke dua Binary Search, data terlebih dahulu diurutkan sebelum dilakukan pencarian. Setelah pencarian hasilnya tetap sama yaitu nilai 77 tidak di temukan dan waktu eksekusi 1400 ns.

```

PS C:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12> cd "c:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12" & java CariNilaiMahasiswa } ; if ($?) { java CariNilaiMahasiswa }
Masukkan nilai yang ingin dicari: 85

=== Linear Search ===
Nilai ditemukan di index: 0
Waktu eksekusi: 400 ns

=== Binary Search ===
Nilai ditemukan di index: 6
Waktu eksekusi: 1500 ns
PS C:\Users\Lenovo\Documents\Semester2\Algoritma\Java\P12>

```

Selain 77 saya juga mencoba mencari nilai 85, Pada metode Linear Search, program langsung menemukan nilai 85 pada indeks ke-0 karena nilai tersebut berada di posisi pertama dalam array. Waktu eksekusi yang dibutuhkan relatif cepat, yaitu sekitar 400 ns, karena pencarian berhenti begitu data ditemukan.

Selanjutnya, pada metode Binary Search, program terlebih dahulu mengurutkan data sebelum melakukan pencarian. Setelah proses pencarian, nilai 85 berhasil ditemukan pada indeks ke-6 (berdasarkan posisi setelah data diurutkan), dengan waktu eksekusi sekitar 1500 ns.

5. Kompleksitas Algoritma

```
1 public class p14 {
2
3     static void tugas1(int[] arr) { // arr.length = n
4         int n = arr.length;
5         int sum = 0;
6
7         // Bagian 1
8         for (int i = 0; i < 100; i++) {
9             sum += i;
10        }
11        System.out.println("Sum konstanta: " + sum);
12
13        // Bagian 2
14        for (int i = 0; i < n; i++) {
15            for (int j = i + 1; j < n; j++) {
16                if (arr[i] == arr[j]) {
17                    System.out.println("Duplikat ditemukan");
18                }
19            }
20        }
21    }
22
23    public static void main(String[] args) {
24        int[] data = {1, 2, 3, 2, 5};
25        tugas1(data);
26    }
27 }
```

```
7         // Bagian 1
8         for (int i = 0; i < 100; i++) {
9             sum += i;
10        }
11        System.out.println("Sum konstanta: " + sum);
12
13        // Bagian 2
```

Pada bagian 1 Kompleksitas: $O(1)$ terdapat Perulangan ini berjalan sebanyak 100 kali, dan jumlah tersebut tetap (konstan), tidak bergantung pada ukuran input (n). Oleh karena itu, waktu eksekusinya tidak berubah meskipun jumlah data bertambah.

```
12
13     // Bagian 2
14     for (int i = 0; i < n; i++) {
15         for (int j = i + 1; j < n; j++) {
16             if (arr[i] == arr[j]) {
17                 System.out.println("Duplikat ditemukan");
18             }
19         }
20     }
21 }
22
```

Dan pada bagian 2 Kompleksitas: $O(n^2)$, Looppertama berjalan sebanyak n kali, Loop kedua juga berjalan hampir n kali untuk setiap i , dan total operasi kira-kira: $n \times n = n^2$